CLA-4 Report: Observations

ML Model Deployment Strategies and Model Serving using FastAPI · Dataset: Titanic (Kaggle) train.csv

Summary

A RandomForest pipeline (imputation, scaling, one-hot encoding) was trained once on 80% of the labelled data and reached **79.89% test accuracy**. The saved pipeline (`model.pkl`) was reused for online, batch and streaming inference and for the FastAPI service.

1. How does Online Inference differ from Batch Inference?

Online inference predicts for a single record on request and returns the result immediately, so latency matters (e.g. a user submitting a form or an API call). Batch inference collects many records (here 20) and scores them together in one call, usually offline or on a schedule, and stores the results (batch_predictions.csv). Batch is more efficient per record and has high throughput, but results are not instant.

2. What is the difference between Streaming Inference and Batch Inference?

Batch inference works on a fixed, finite set of records that already exists. Streaming inference handles records continuously as they arrive, one (or a few) at a time, with low delay, as in our simulation that processed one record per second. Streaming gives near-real-time results on unbounded data; batch gives results only after the whole set is collected.

3. Why is FastAPI used for model serving?

FastAPI is fast (ASGI, async-capable) and simple to write. Pydantic gives automatic request validation, so bad input (wrong Pclass, negative age) is rejected with a clear 422 error before reaching the model. It auto-generates interactive Swagger UI (/docs) and OpenAPI docs for testing, uses Python type hints, and lets any client (web, mobile, other services) use the model over HTTP.

4. Why should the same trained model be reused for all inference strategies?

Reusing one saved pipeline guarantees consistent predictions regardless of how data arrives, and avoids training-serving skew because the same preprocessing is bundled with the model. It also saves compute, makes results reproducible and comparable, makes versioning and updates easy (replace one file), and ensures evaluation metrics apply to every deployment path.

Results at a glance

- Test accuracy: 79.89% (179 unseen records)
- Online: 1 record; Batch: 20 records saved to CSV; Streaming: 10 records at 1 s intervals
- API: /health 200; valid /predict 200; invalid /predict 422